



4WD Torque Vectoring

A Modular Control Development Platform

A specification for a modular simulation platform to develop, compare, and validate control architecture for a four-wheel-drive hub-motor Formula Student car — seeded by the Year 4 yaw-control thesis.

UNIVERSITY OF STRATHCLYDE MOTORSPORT

Based on: *Modelling and Simulation of Yaw Moment-Based Torque Vectoring Control for a Formula Student Vehicle* (BEng thesis, April 2026)

Contents

0	Executive summary	3
1	Diagnosis — why the thesis is a seed, not a conclusion	4
2	The 4WD control problem	5
2.1	Over-actuation: from tuning a controller to choosing an architecture	5
2.2	The architecture space (allocation × controller)	6
2.3	Longitudinal control as the next layer	8
3	The configuration ladder: RWD-single → RWD-twin → 4WD	9
4	Platform architecture & modularity	10
4.1	Decoupled layers and interface contracts	10
4.2	What incremental upgrade looks like in practice	11
5	Thesis vs. target: system structure	12
6	Implementation roadmap	13
7	Metrics for 4WD control development	18
8	Modular extension & roadmap	19

How to read this document

Sections 1–3 set up the problem: why a 4WD hub-motor car needs a purpose-built control development tool, and how the team's three vehicle configurations fit into one platform. Sections 4–8 are the platform itself — its modular architecture, the build, the metrics, and how it grows. The platform is deliberately **architecture-neutral**: it exists to let the team develop and compare control strategies, not to prescribe one. The code excerpts are illustrative scaffolding; the actual repository files are produced separately.

0 Executive summary

The team's target is a **four-wheel-drive, hub-motor** Formula Student car: four independently-controlled wheel torques. That hardware does not yet exist, but the control system that will run it has to be designed, compared against alternatives, and de-risked long before the first motor is bolted on. This document specifies a **modular simulation platform** to do exactly that.

The Year 4 thesis is the seed. It built a yaw-rate-tracking study for a single-actuator car and found that a PI and a fuzzy controller performed almost identically. That result is the starting point for two reasons: it shows the modelling scaffold works, and — more importantly — it shows that a single-track, single-actuator model cannot tell control strategies apart. A 4WD hub-motor car is the opposite kind of system, and needs a different kind of tool.

Three ideas define the platform:

- **4WD makes control architecture the central problem.** With four independent torque sources, there are infinitely many ways to produce a given yaw moment. The engineering question stops being 'tune a controller' and becomes 'choose an architecture' — a *controller* (PI / fuzzy / MPC / sliding-mode for the yaw layer) paired with an *allocation strategy* (how the demanded moment is split across four motors). The platform treats these two as the co-equal halves of architecture and lets the team evaluate combinations.
- **One model, three configurations.** The team's current car (RWD, single motor), the data-source car (RWD, twin hub motors), and the target (4WD, four hub motors) are three settings of one configurable model. The simpler configurations are where a controller is validated against real data before being extrapolated to the 4WD target.
- **Modularity is a requirement, not a nicety.** Vehicle model, controller, allocator, and powertrain are separate blocks with defined interfaces, so any one can be upgraded — a better tyre model, a new controller, an added longitudinal-control module — without disturbing the others. The platform is built to be handed on and incrementally improved.

What the finished platform does

A controls engineer picks a controller and an allocation strategy, picks a vehicle configuration (RWD-single, RWD-twin, or 4WD), runs a standard manoeuvre, and reads a metric set that exposes yaw tracking, per-wheel tyre utilisation, allocation behaviour, and energy use. They can then swap any block and re-run. That is a tool for guiding 4WD control-architecture selection — the decision the team faces — rather than a fixed answer to it.

Build status — 23 June 2026

This document describes the target platform. **Implemented (tag v0.3-baseline-loop):** the full modular skeleton (six interface buses, five referenced-model blocks plus a manoeuvre generator, wired in a coherent closed loop under Git source control); a linear single-track plant; the understeer reference; a PI controller; a baseline rule-based allocator with the configuration ladder; and a metric harness (RMSE / IAE / peak yaw-rate tracking). The loop runs and reproduces the thesis behaviour — the thesis now lives inside the modular platform. **Not yet built:** the double-track plant (§5), the constraint-aware allocator (§2.2), the controller/allocator comparison grid, powertrain realism, twin-motor correlation, and longitudinal control (§2.3). See the companion implementation tracker for the live checklist.

1 Diagnosis

Why the thesis is a seed, not a conclusion

The thesis asked a well-posed question — *do a PI and a fuzzy controller behave differently when tracking a reference yaw rate?* — and answered it rigorously: within its modelling framework, they do not. The metrics were near-identical. Taken at face value that reads like a dead end. Reframed for a 4WD programme, it is the opposite: it is precisely the evidence that motivates building a proper development platform.

The reason the controllers tied is that the **test rig could not tell them apart**. A linear, single-track, single-actuator model has no mechanism that rewards one strategy over another. That limitation is mild for the RWD car the thesis modelled. For a 4WD hub-motor car it is disqualifying — the model is structurally incapable of representing the very things that make 4WD control interesting:

What the thesis model lacks	Why it disqualifies the model for 4WD control study
One actuator, not four	The thesis applies a single yaw moment through one effective actuator. A 4WD car has four independent wheel torques. The central 4WD problem — how to <i>distribute</i> torque across wheels — simply does not exist in a single-actuator model, so the model cannot be used to study it.
No per-wheel tyre behaviour	A single-track (bicycle) model lumps left and right tyres together. Torque vectoring works by deliberately making the left and right tyres do different things; a model that cannot separate them cannot represent the mechanism it is meant to study.
Linear tyres, no saturation	With $F_y = C \cdot \alpha$ and no friction limit, there is no grip ceiling to manage. Allocation strategy only starts to matter when tyres saturate and the controller must decide which wheel to lean on — exactly what a linear model removes.
No load transfer	Equal vertical load on every tyre means the per-wheel grip budget never changes. Real cornering shifts load, and a good allocator exploits that asymmetry. Without it, all four motors look interchangeable.

So the thesis is not the answer to the team's question — it is the proof that the team needs a better question and a better tool. The valuable inheritance from it is not the 'PI $\approx$ fuzzy' conclusion but the working scaffold beneath it: a reference generator, a controller interface, a metric harness, and a telemetry pipeline. The platform keeps that scaffold and rebuilds the plant and the actuation around the 4WD target.

The re-frame in one line

'The two controllers were identical' $\rightarrow$ 'A single-actuator model cannot distinguish control strategies at all — so to develop control for a four-actuator car, we must build a model that can.' Everything that follows is that model and the tooling around it.

2 The 4WD control problem

Four independent hub motors change the nature of the control task. This section sets out why, lays out the space of architectures the platform must be able to represent, and frames the two co-equal halves of an architecture — controller and allocator — that the team will develop and choose between. It is deliberately even-handed: the platform's role is to let the team evaluate these options, not to pick for them.

2.1 Over-actuation: from tuning a controller to choosing an architecture

A single-motor RWD car has essentially one lateral-control actuator: the yaw moment produced by the powertrain. The control problem is well-posed and one-dimensional — pick a controller, tune it. A 4WD hub-motor car has **four independent torque inputs** but still only needs to deliver a small number of vehicle-level targets (a yaw moment, a total longitudinal force). There are far more actuators than targets — the system is **over-actuated** — which means there are infinitely many ways to produce any given demand.

That redundancy is the whole opportunity, and the whole difficulty. The same yaw moment can be made by adding torque on the outer wheels, removing it from the inner wheels, or any blend; each choice uses the tyres and the motors differently and costs different energy. Deciding *which* of the infinite solutions to use is the **allocation problem**, and it does not exist on a single-actuator car. This is why, for 4WD, the engineering effort shifts from tuning a single loop to designing an architecture: a layer that decides the vehicle-level demand, and a layer that distributes it across the wheels.

The defining shift

Single-motor RWD: **one** way to make a yaw moment — the question is how well you track. 4WD hub motors: **infinitely many** ways — the question is which one you choose, and how the choosing is structured. The platform exists to explore that choice.

2.2 The architecture space (allocation × controller)

An architecture is a pairing of two decisions. The **controller** decides *what* vehicle-level correction is needed (e.g. how much yaw moment to add to track the target). The **allocator** decides *how* to realise it across the four motors. Both halves admit several established options; the platform must represent the full grid so the team can compare combinations.

Controller layer (the 'what')

Controller	Character / trade-off
PI / PID	Simple, transparent, easy to tune and to trust. The thesis baseline. Limited at the nonlinear limit and cannot anticipate constraints.
Fuzzy (Mamdani)	Encodes engineer intuition as rules; smooth, no explicit model needed. Hard to tune systematically and to prove robust.
Sliding-mode	Robust to model error and disturbance — attractive when the plant is uncertain. Can chatter; needs care at the actuator.
MPC (model-predictive)	Handles constraints and over-actuation natively — can fold allocation and tyre/motor limits into one optimisation. Highest complexity and compute; needs a good model.

Allocation layer (the 'how')

Allocation strategy	Character / trade-off
Fixed rule / heuristic	Pre-set split (e.g. proportional to load or a fixed map). Cheap, predictable; ignores real-time tyre state.
Pseudo-inverse / weighted	Least-effort distribution from control-allocation theory. Cheap, smooth; respects simple weightings but not hard limits.
Constrained optimisation	Minimise an objective (tyre use, energy) subject to friction-circle and motor limits, solved each step. Powerful and limit-aware; heavier to run and tune.
Independent-wheel	Each wheel controlled to its own target (e.g. slip). Maximal authority; shifts the burden onto coordinating the per-wheel targets.

Tables 2.1 / 2.2 — The two halves of an architecture. The platform must let any controller pair with any allocator, because the right combination depends on the car, the tyres, and the team's compute budget — which is what the platform is for measuring.

Two ways to wire the architecture

The same controller and allocator options can be organised in different topologies. The two most relevant are a **hierarchical** split (a controller produces a vehicle-level demand, a separate allocator distributes it) and a **centralised** scheme (one optimisation maps targets straight to four wheel torques). The platform should represent both; the diagram contrasts them.



Figure 2.1 — Two architecture topologies the platform must support. Hierarchical keeps controller and allocator as separate, swappable blocks (easier to develop incrementally); centralised fuses them into one constraint-aware optimisation (more capable, more complex). The platform stays neutral — both are configurations to be evaluated.

The hierarchical topology maps naturally onto the modular design of Section 4: controller and allocator are distinct blocks behind a defined interface, so each can be swapped without the other. The centralised topology trades that modularity for capability. Neither is ‘right’ — which is exactly why the platform must run both and let the metrics decide.

2.3 Longitudinal control as the next layer

Four hub motors do not only enable yaw control; they enable independent **longitudinal** control of each wheel — traction control and anti-lock braking realised in software, per wheel, through the same actuators. This is a natural extension of the same allocation machinery: once the platform can distribute torque to influence yaw, distributing it to manage per-wheel slip is the same kind of problem with a different target.

The platform scopes longitudinal control as the **second layer**, developed once the yaw layer works — and crucially, it is the first real test of the modular design. Adding traction/ABS should mean adding a module that shares the allocator and the plant, not rewriting the yaw controller. If the architecture is right, the longitudinal layer slots in against the existing interfaces; if it forces changes elsewhere, the interfaces were drawn in the wrong place. In that sense the longitudinal layer is both a capability and a design test.

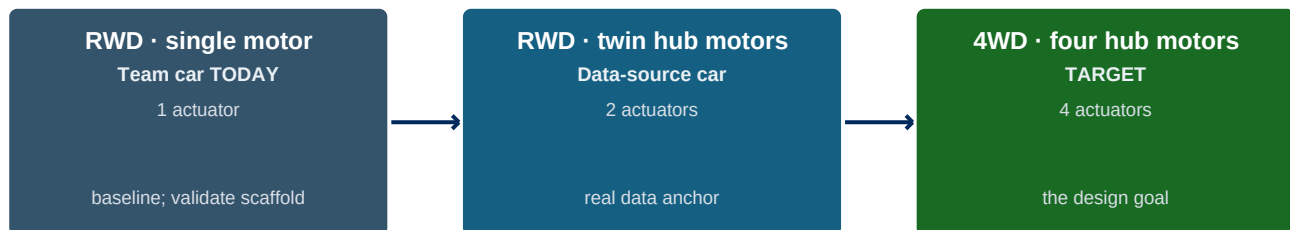
Why include it now

Longitudinal control shares the actuators, the tyre model, and the allocator with yaw control. Designing the platform with it in mind — even before building it — ensures the interfaces are drawn so that the combined-slip tyre behaviour and the shared torque budget are representable. Retrofitting it later into a yaw-only design is far harder.

3 The configuration ladder

RWD-single → RWD-twin → 4WD as one configurable model

The team is not starting from nothing, and the target is not reached in one jump. Three vehicle configurations are in play, and the platform's strength is that they are not three projects but **three settings of one model** — selected by changing the actuator configuration, nothing else.



same model · same controllers · only the actuator configuration changes

Figure 3.1 — The configuration ladder. Each rung is a value of the actuator configuration in the vehicle config; the controller and allocator code above it is unchanged.

The ladder gives the development effort a spine. A controller can be brought up on the single-motor configuration (where behaviour is simplest and matches the team's current car), exercised on the twin-motor configuration (where genuine left/right torque vectoring first appears, and where real data exists to check against), and only then extrapolated to the four-motor target. Each rung de-risks the next.

The validation anchor

One rung carries particular weight: the twin-hub-motor RWD car is the configuration for which the team has access to **real telemetry**. That makes it the natural correlation anchor — the configuration where simulated behaviour can be checked against measured behaviour before any conclusion is trusted. It does not validate the 4WD target directly, but a model that correlates on the twin-motor car, and is then changed only by adding two more identical actuators, is a far more credible basis for 4WD development than a model validated against nothing. The platform should treat this data path as a first-class feature — without letting it dominate the architecture work, which is where most of the value is created.

Why this matters for credibility

Developing 4WD control purely in simulation invites the question 'how do you know the model is right?'. The twin-motor data answers it for the parts that can be measured now, so the extrapolation to four motors rests on correlated foundations rather than assumption.

4 Platform architecture & modularity

The team's explicit requirement is a platform that can be **incrementally updated** — upgraded vehicle models, new controllers, new modules — over several seasons and several student generations. That requirement is met by one architectural commitment: clean separation of concerns behind defined interfaces. This section is the design that makes everything else swappable.

4.1 Decoupled layers and interface contracts

The signal chain is split into independent blocks, each with a defined input/output contract. As long as a block honours its contract, its internals can be replaced freely — a new controller, a higher-fidelity tyre model, a different allocator — without touching its neighbours.

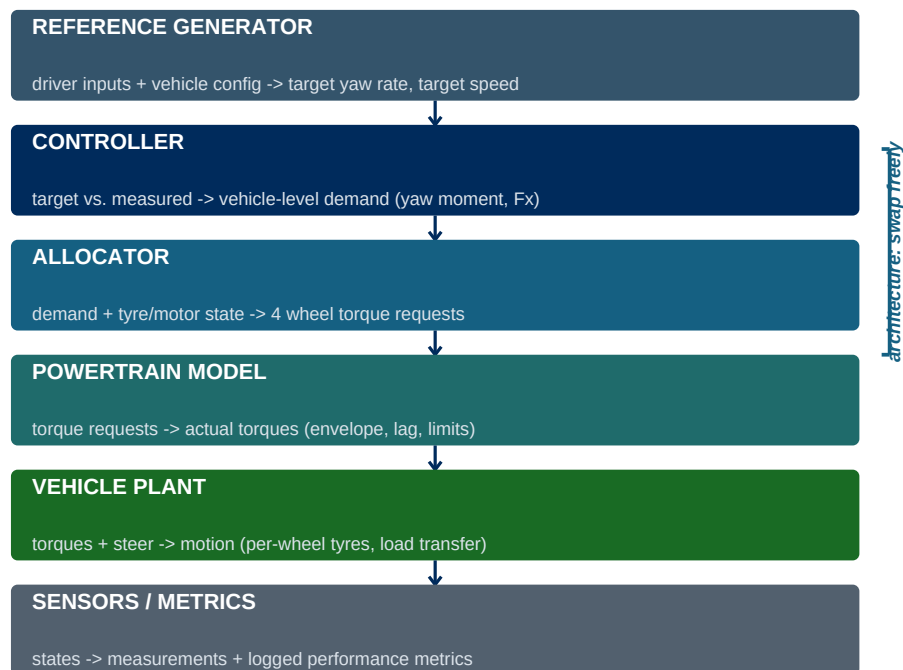


Figure 4.1 — The platform as a stack of contracted blocks. The controller and allocator (the architecture) sit between a fixed reference interface above and a fixed powertrain interface below, so they can be swapped without disturbing the rest. The vehicle plant and powertrain can be upgraded in fidelity independently.

The interface contracts

Modularity is only real if the interfaces are written down. The platform defines each boundary as a fixed signal contract; the table is the contract set a new module must honour.

Interface	Carries	So you can swap...
Reference → Controller	Target yaw rate, target speed, current states	the reference model (e.g. add a friction-limited or nonlinear target) without touching the controller
Controller → Allocator	Vehicle-level demand: yaw moment M_z , longitudinal force F_x	any controller (PI/fuzzy/MPC/SMC) for any other, as long as it outputs the same demand
Allocator → Powertrain	Four wheel-torque requests	any allocation strategy (rule/pseudo-inverse/optimisation) without changing the controller or the motors
Powertrain → Plant	Four actual wheel torques	the motor/inverter model fidelity (ideal → envelope+lag → thermal) independently
Plant → Sensors	Full vehicle + per-wheel state	the vehicle model (bicycle → double-track → higher fidelity) without affecting the controller above

Table 4.1 — Interface contracts. Each row is a boundary that, once fixed, lets the block on one side be replaced without disturbing the other.

4.2 What incremental upgrade looks like in practice

The test of the design is whether common upgrades touch only one block. They do:

- **New controller** (e.g. add MPC): implement against the Reference→Controller and Controller→Allocator contracts; nothing else changes. Compare it to PI by switching one config field.
- **Better vehicle model** (bicycle → double-track with load transfer): replace the Plant block; the controller and allocator see the same interface and are unaffected.
- **New allocation strategy**: swap the Allocator block; the controller still emits the same demand and the motors still receive four torque requests.
- **Longitudinal-control module** (traction/ABS): added alongside the yaw controller, sharing the allocator and plant via the existing contracts — the worked example of modular growth from Section 2.3.
- **Higher-fidelity powertrain** (add thermal derating): replace the Powertrain block internals; the allocator's torque requests are unchanged.

The modularity payoff

Every upgrade above is a one-block change. That is what 'incrementally updatable' means concretely: a new student can improve one part of the platform in a single term without needing to understand — or risk breaking — all the others.

5 Thesis vs. target: system structure

A side-by-side of the system as the thesis built it and as the platform must become it for 4WD control development. Each change traces to the diagnosis in Section 1 — the model must represent four actuators and per-wheel tyre behaviour — or to the modularity requirement of Section 4.

Axis	Thesis (as-built)	Target (4WD platform)	What it unlocks
Purpose	Compare two controllers on one car	Develop & compare control architecture for a 4WD hub-motor car	Guides architecture selection, the team's real decision
Actuation	Single effective yaw actuator	Four independent wheel torques + allocator	Makes the allocation problem — the core of 4WD — representable
Plant	Linear single-track, symmetric	Double-track, per-wheel nonlinear tyres, load transfer	Lets strategies be told apart; gives allocation something to exploit
Input	One telemetry file	Standard manoeuvres + telemetry replay for correlation	Reproducible development; data anchor on the twin-motor rung
Config	Fixed RWD car	Actuator configuration selectable (ladder of Sec 3)	One model serves all three team configurations
Structure	Monolithic model	Contracted, swappable blocks (Sec 4)	Incremental upgrade; new controllers/modules drop in
Metrics	IAE/RMSE/peak (tracking)	+ per-wheel tyre use, allocation, energy, constraints	Architecture-relevant; exposes why one strategy beats another

Table 5.1 — Thesis system vs. the 4WD development platform. The shaded column is the target. The two structural changes — four actuators and a double-track plant — are what make the rest meaningful.

The single most consequential row is **Actuation**. Adding an allocator and moving from a single yaw input to four wheel torques is the change that turns the thesis's yaw-tracking study into a 4WD control platform. Everything in the roadmap is sequenced around getting that actuation chain — controller, allocator, four-motor plant — standing first, then raising its fidelity.

6 Implementation roadmap

A staged build, sequenced so the platform is usable after each stage and so the 4WD actuation chain stands early. The allocator is a first-class block from the start, and the actuator configuration is selectable from Stage 1 — the ladder of Section 3 is built in, not bolted on. Effort and priority flags are the sub-team's to re-weight.

Stage 1 · Self-contained repo + configurable actuation skeleton

Effort: Low–Med Priority: Do first

Establish the modular skeleton with the actuator configuration selectable from day one. Strip the thesis's hard-coded paths and telemetry-coupled initialisation; stand up the block interfaces of Section 4 even if early blocks are placeholders.

- Relative paths, a single `run_study.m` entry point, README and HANDOVER, and version control (the binary-.slx conventions matter more here than ever — multiple students will edit this).
- A `vehicleConfig` struct carrying geometry, mass, tyres, and an `actuatorConfig` field selecting RWD-single / RWD-twin / 4WD.
- Placeholder allocator and controller blocks honouring the Section 4 contracts, so the chain runs end-to-end before any block is sophisticated.

```
%% vehicleConfig — single source of truth, actuator-agnostic
cfg.geom = struct('L',1.59,'a',0.816,'tr',1.23,'h',0.30); % h *** MEASURE ***
cfg.mass = struct('m',255,'Iz',100); % Iz *** estimate ***
cfg.tyre = struct('model',"linear",'mu',1.3,'Cf',46e3,'Cr',50e3);

% --- the ladder lives here: one field selects the configuration ---
cfg.actuator = "RWD_twin"; % "RWD_single" | "RWD_twin" | "AWD_4"
cfg.arch = struct('controller',"PI", 'allocator',"pseudo_inverse", ...
    'topology',"hierarchical");
```

Stage 2 · Double-track plant with per-wheel tyres

Effort: Med-High Priority: Core

The structural fidelity jump. Replace the single-track plant with a double-track (four-corner) model so left and right wheels carry independent loads and forces — the prerequisite for any torque-vectoring study. Add lateral (and later longitudinal) load transfer and a saturating per-wheel tyre.

```
% per-corner vertical load (lateral transfer; add longitudinal later)
Fz_fl = m*g*(b/L)/2 - dFz_lat_f;   Fz_fr = m*g*(b/L)/2 + dFz_lat_f;
Fz_rl = m*g*(a/L)/2 - dFz_lat_r;   Fz_rr = m*g*(a/L)/2 + dFz_lat_r;

% per-wheel saturating tyre (Fiala/MF) — each corner independent
for k = ["fl","fr","rl","rr"]
    Fy(k) = tyre_fy(alpha(k), Fz(k), C(k), mu);   % clips at mu*Fz
end
```

Why double-track is non-negotiable for 4WD

Torque vectoring is, by definition, making the left and right wheels do different things. A single-track model averages them away. No allocation strategy can be meaningfully studied until the plant resolves the four corners separately — this is the stage that makes the platform a 4WD tool rather than a yaw-tracking demo.

Stage 3 · Allocator block + actuator topologies

Effort: Med-High Priority: Core

Build the allocator as a first-class, swappable block implementing the Section 4 contract: in comes a vehicle-level demand (yaw moment, longitudinal force), out go four wheel-torque requests. Provide at least two strategies so comparison is possible from the start, and make the actuator topology (how many/which wheels are driven) read from the config.

```
function T = allocate(demand, state, cfg)
% demand: [Mz; Fx] -> T: [T_fl; T_fr; T_rl; T_rr]
    switch cfg.arch.allocator
        case "rule" % fixed/heuristic split
            T = rule_split(demand, cfg);
        case "pseudo_inverse" % least-effort weighted distribution
            T = Binv(cfg) * demand;
        case "optimisation" % min tyre-use s.t. friction & motor limits
            T = solve_qp(demand, state, cfg);
    end
    T = apply_topology(T, cfg.actuator); % zero undriven wheels (RWD configs)
end
```

The `apply_topology` call is the ladder in action: for `RWD_single` and `RWD_twin` it zeroes the undriven wheels, so the same allocator and controller run unchanged across all three configurations.

Stage 4 · Manoeuvres, controllers & the metric harness

Effort: Med Priority: High

Make the platform exercise itself. Standard manoeuvres (step steer, skidpad, slalom, ramp steer) drive reproducible tests; a controller-agnostic metric function (Section 7) judges any controller/allocator pairing by the same yardstick. Port the thesis PI and fuzzy controllers as the first two controller-block instances.

```
%% run_study.m — exercise one architecture on one configuration
cfg = vehicleConfig();
cfg.actuator      = "AWD_4";           % ladder rung
cfg.arch.controller = "PI";            % PI | Fuzzy | MPC | SMC
cfg.arch.allocator = "pseudo_inverse"; % rule | pseudo_inverse | optimisation
man = manoeuvre("ramp_steel", cfg);
out = sim("YawTV_4WD.slx");
M    = metrics(out, cfg);              % yaw + per-wheel + allocation + energy
disp(struct2table(M));
```

The comparison the platform is built to produce

Loop over architectures — {PI, Fuzzy, MPC} × {rule, pseudo-inverse, optimisation} — on a fixed manoeuvre and configuration, and tabulate the metric set. That grid is the evidence the team uses to select a 4WD control architecture.

Stage 5 · Powertrain realism + twin-motor correlation

Effort: Med Priority: Med-High

Close the loop on credibility. Add a configurable powertrain block (torque–speed envelope, actuator lag, power and regen limits) between allocator and plant, and correlate the twin-motor configuration against the real telemetry from the data-source car. Once the model tracks measured behaviour on that rung, the 4WD extrapolation rests on validated foundations.

- Powertrain block honouring the Section 4 contract: requested torques in, realistic torques out, so no other block changes.
- Telemetry-replay mode feeding measured steering/speed through the twin-motor configuration; overlay simulated vs. measured yaw rate.
- Flag the parameters that still need measurement — CG height h , yaw inertia I_z , a proper tyre fit from the TTC data — as the correlation backlog.

7 Metrics for 4WD control development

Metrics for a 4WD platform must do more than score tracking — they must expose *why* one architecture beats another, which for an over-actuated car means looking at how the four actuators are used, not just whether the yaw target was met. The set below groups into tracking, the per-wheel and allocation metrics that are specific to 4WD, and energy — which hub motors make a real currency.

Metric	Group	What it tells the developer
RMSE / IAE yaw rate	Tracking	How tightly the architecture holds the target yaw rate.
Yaw response time, overshoot	Tracking	Transient quality — turn-in and settling.
Per-wheel tyre utilisation $ F /(\mu F_z)$	4WD	How close each tyre runs to its limit — the grip headroom an allocator is managing.
Allocation distribution / balance	4WD	How the demanded moment is split across wheels — even, outer-biased, saturating one corner.
Constraint-satisfaction margin	4WD	Whether torque requests stay within friction-circle and motor limits, or saturate.
Control effort $\ T\ $ per wheel	4WD	Actuator workload and inverter duty per motor.
Peak lateral accel a_y	Tracking	Grip-limited cornering capability achieved.
Energy per manoeuvre	Energy	Total electrical energy — efficiency cost of the strategy, which differs between allocations producing the same yaw.
Yaw-moment efficiency (Mz per kWh)	Energy	How much useful yaw authority each strategy buys per unit energy.

Table 7.1 — Metric set. The **4WD** group is what a single-actuator study cannot produce and what makes architecture comparison possible; the **Energy** group matters because two allocations can make the identical yaw moment at very different efficiency.

Reading the per-wheel metrics

Two architectures may post identical yaw-tracking RMSE yet differ completely in tyre utilisation and energy — one running a corner to saturation, the other spreading the load. The per-wheel and energy metrics surface that difference, which is precisely the information needed to choose between strategies that look equal on tracking alone.

8 Modular extension & roadmap

The platform is designed to outlive any one student. This closing section sets the sequence for building it and the directions it is built to grow in — each a one-block addition against the interfaces of Section 4.

8.1 Suggested sequencing

Order	Task	Effort	Leaves you with
1	Repo skeleton + configurable actuation (Stage 1)	Low–Med	A version-controlled, runnable chain with the config ladder built in.
2	Double-track plant + per-wheel tyres (Stage 2)	Med–High	A model that can actually represent torque vectoring.
3	Allocator block + topologies (Stage 3)	Med–High	The 4WD actuation chain standing; strategies comparable.
4	Manoeuvres, controllers, metrics (Stage 4)	Med	Architecture comparison grid producing real evidence.
5	Powertrain realism + twin-motor correlation (Stage 5)	Med	A credible, partially-validated model for 4WD extrapolation.
6	Longitudinal control module (Sec 2.3)	Med–High	Traction/ABS layer; first proof of modular growth.
7	Advanced architectures (MPC, optimisation allocation)	High	Constraint-aware control; the upper end of the comparison grid.

Table 8.1 — Build order. Tasks 1–3 stand up the 4WD chain; 4–5 make it produce trustworthy evidence; 6–7 are the first major modular extensions.

8.2 Directions the platform is built to grow

■ Richer controllers

MPC and sliding-mode against the same controller interface — the comparison grid simply gains columns. MPC in particular can absorb the allocator into one constraint-aware solve (the centralised topology of Section 2.2).

■ Longitudinal & combined-slip

Traction/ABS as a module sharing the allocator and plant, plus a combined-slip tyre so lateral and longitudinal demands interact realistically.

■ Higher plant fidelity

Suspension/roll dynamics, longitudinal load transfer, aero, and configurable roll-stiffness distribution — each a Plant-block upgrade behind the same interface.

■ Parameter sweeps & robustness

Batch runs over vehicle and powertrain configs to test whether an architecture is robust or tuned to one car — trivial once the config layer exists.

■ Toward HIL / digital twin

With the twin-motor correlation in place, progress toward a hardware-in-the-loop setup and a digital twin of the actual 4WD car as it is built.

The platform in one sentence

A modular, configuration-aware simulation in which any controller can be paired with any allocator on any of the team's three vehicle configurations, judged by metrics that expose how the four actuators are used — a tool to develop, compare, and de-risk the 4WD car's control architecture before the hardware exists, and to keep improving it season after season.

This is a living working document. As the platform is built, Sections 4 and 6 should be updated to reference the actual blocks, interface signal lists, and conventions the sub-team adopts — the document and the codebase should evolve together.